

Project Track 3 – E-Commerce Order Management System

Project Report

Prepared by: Merna Abd El Fatah Soliman

1. Project Overview

This project presents an E-Commerce Order Management System designed to manage products, suppliers, customers, orders, order items, payments, shipments, and inventory. The system uses SQL Server to ensure data integrity, automate business rules, process orders, manage stock, and support sales analysis.

Project Objectives

- Manage products, categories, and suppliers.
- Manage customers and their orders.
- Maintain accurate product stock levels.
- Track payments and shipments.
- Apply business rules using functions, stored procedures, and triggers.
- Improve query performance using indexes.
- Provide data for sales and customer analysis.

2. Business Rules & Assumptions

Business Rules

- Each product belongs to one category and one supplier.
- Each customer can place multiple orders.
- Each order contains one or more order items.
- Product stock decreases when an order item is created.
- An order can have multiple payment records.
- Orders follow the statuses: Pending, Processing, Shipped, Delivered, and Cancelled.
- Products referenced in historical orders cannot be deleted.
- Payments cannot exceed the remaining order balance.
- Creating a shipment automatically changes the order status to Shipped.
- Low-stock products are identified when `StockQuantity < 10`.

Assumptions

- Product and customer IDs are unique.
- Product names and customer emails are unique.
- Prices, quantities, stock values, and payment amounts must be valid positive values.
- The database uses SQL Server.
- The system uses transactions and error handling to maintain data consistency.

3. Database Design – ERD

The database was designed using a relational model to represent the main entities and relationships of the E-Commerce Order Management System. The design includes customers, orders, order items, products, categories, suppliers, payments, shipments, and the custom coupon feature.

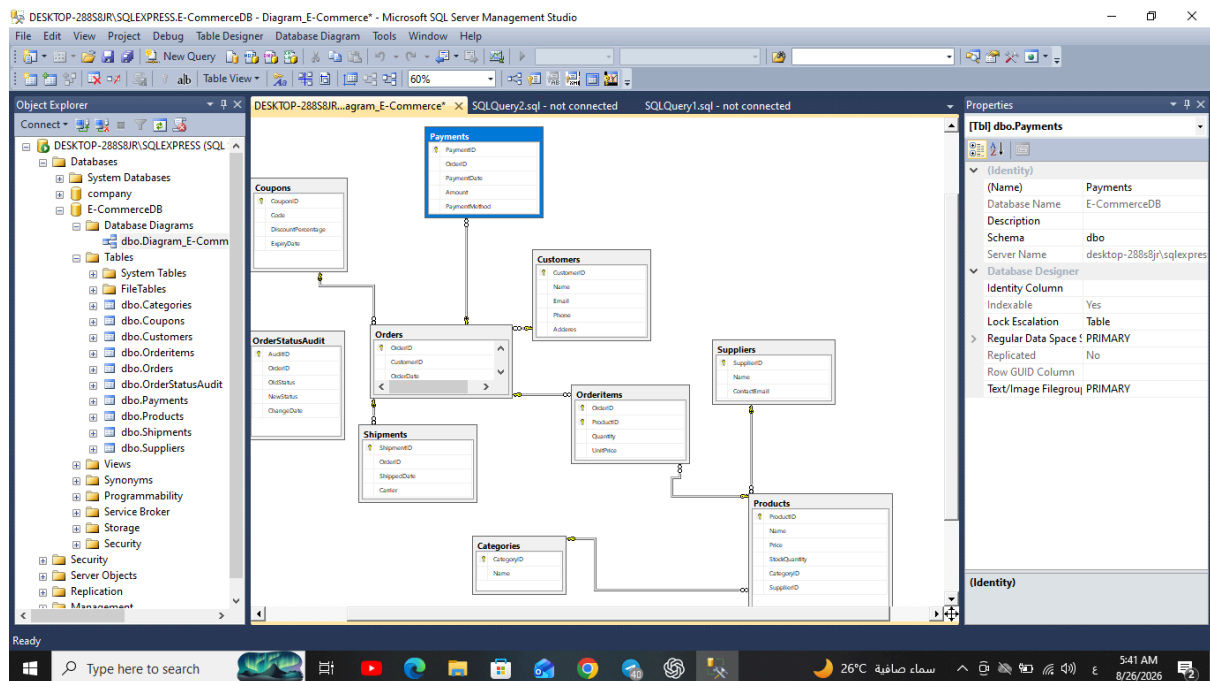


Figure 1. Entity Relationship Diagram (ERD) of the E-Commerce Order Management System.

4. Data Import & Seeding

Product catalog data was imported into the **Products** table using the SQL Server Import and Export Wizard from a CSV file. The imported data was successfully appended to the existing product records in the database.

The database was also seeded with data for customers, products, orders, order items, payments, shipments, categories, suppliers, and coupons to support testing of the system functionality.

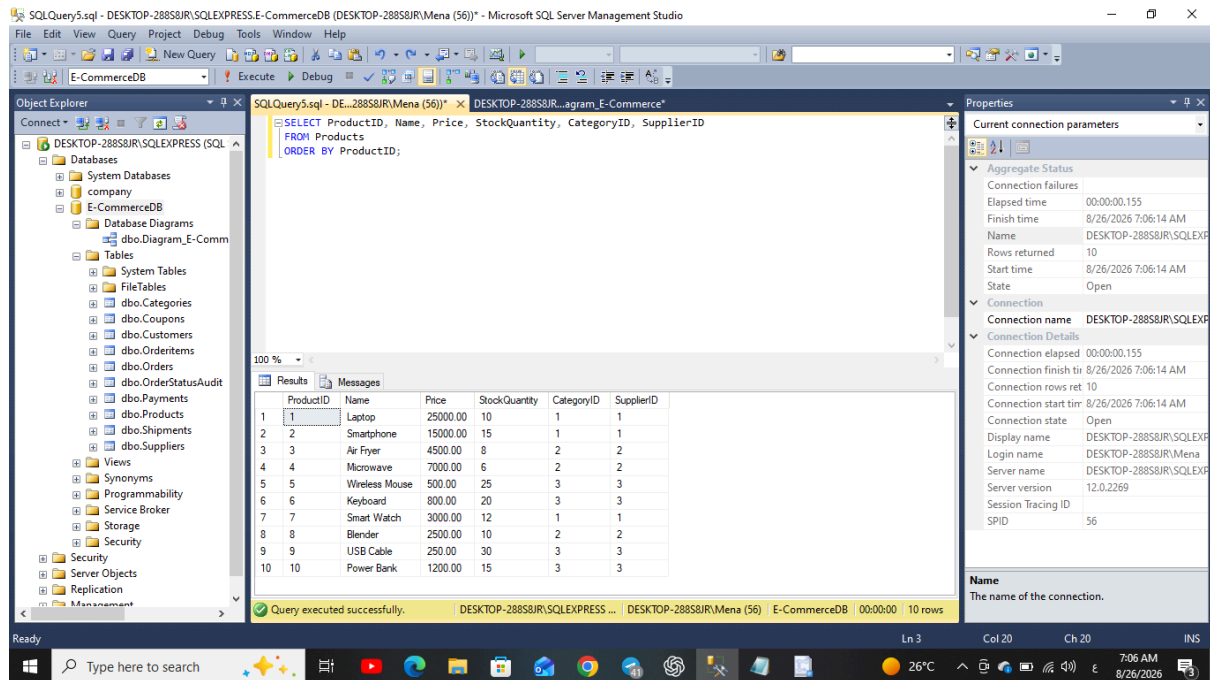


Figure 2. Product records after successful CSV import using SQL Server Import and Export Wizard.

5. DQL Queries

The system includes several SQL queries to support sales analysis, inventory monitoring, revenue analysis, customer spending, and multi-table reporting.

Query 1 – Sales Revenue and Top Spending Customers

Calculates sales revenue by product category and identifies customers with the highest spending.

Query 2 – Low-Stock Products

Lists products with low stock levels and displays their supplier contact information.

Query 3 – Month-over-Month Revenue

Uses CTEs and date functions to calculate and compare monthly revenue.

Query 4 – Customer Running Expenditure

Uses the `SUM() OVER()` window function to calculate the running total expenditure for each customer.

Query 5 – Multi-Table Order Report

Uses multiple JOIN operations to link Customers, Orders, OrderItems, Products, Shipments, and Payments.

These queries provide useful information for inventory control, revenue analysis, customer analysis, and order tracking.

6. Database Views

Five database views were implemented to simplify data retrieval and provide reusable summaries for the E-Commerce Order Management System.

6.1 vw_OrderSummary

Summarizes order information including order totals, item counts, customer details, and the current order status.

6.2 vw_LowStockProducts

Displays products with stock levels below the defined low-stock threshold and helps identify products requiring reorder.

6.3 vw_SalesByCategory

Aggregates sales revenue, units sold, and average selling price for each product category.

6.4 vw_CustomerLifetimeValue

Calculates the total number of orders and cumulative revenue generated by each customer.

6.5 vw_PendingShipments

Lists processed orders that are awaiting shipment dispatch and provides the related carrier information.

These views improve readability, reduce repeated query logic, and support reporting and analysis.

7. Database Functions

Five SQL functions were implemented to support order calculations, customer analysis, stock validation, order history, and product sales analysis.

7.1 fn_GetOrderTotal

A scalar function that calculates the total monetary value of a specific order based on its order items.

7.2 fn_GetCustomerTotalSpent

A scalar function that calculates the customer's total lifetime spending.

7.3 fn_CheckStockAvailability

A scalar function that checks whether the requested quantity of a product is available in stock.

7.4 fn_GetCustomerOrders

A table-valued function that returns the complete order history of a specified customer.

7.5 fn_GetTopSellingProducts

A table-valued function that returns the top N best-selling products based on the quantity sold.

These functions provide reusable business logic and simplify common calculations and data retrieval operations.

8. Stored Procedures

Five stored procedures were implemented to manage the main order processing and payment operations. TRY...CATCH error handling and transactions were used where required to maintain data consistency.

8.1 sp_CreateOrder

Creates a new order and its line items after validating the customer, products, and available stock.

8.2 sp_UpdateOrderStatus

Updates the lifecycle status of an order while validating the allowed order statuses.

8.3 sp_ProcessPayment

Records a payment for an order and checks whether the order balance has been fully or partially settled.

8.4 sp_CancelOrder

Cancels an order, restores the related product stock quantities, and updates the order status to **Cancelled**.

8.5 sp_ApplyDiscountCoupon

Validates a coupon code and applies the corresponding discount to the order according to the defined business rules.

These procedures centralize important business operations and help maintain consistency and controlled access to the database.

9. Database Triggers

Five triggers were implemented to automate important business rules and maintain data integrity.

9.1 trg_UpdateStockOnOrder

Automatically reduces the available product stock when order items are inserted.

9.2 trg_PreventProductDelete

Prevents deletion of products that are referenced by historical orders.

9.3 trg_AuditOrderStatus

Records changes in order status in the `OrderStatusAudit` table for tracking and auditing purposes.

9.4 trg_CheckPaymentAmount

Prevents a payment from exceeding the remaining balance of the related order.

9.5 trg_AutoUpdateShipmentStatus

Automatically changes the order status to `Shipped` when a shipment record is created.

These triggers automate business rules, improve data integrity, and reduce the need for manual validation.

10. Indexes & Execution Plans

Five indexes were implemented to improve query performance and optimize data retrieval.

10.1 Orders(CustomerID)

A non-clustered index was created on `Orders(CustomerID)` to improve queries that retrieve orders for a specific customer.

10.2 OrderItems(ProductID)

A non-clustered index was created on `OrderItems(ProductID)` to improve product-related order item searches.

10.3 Products(CategoryID)

A non-clustered index was created on `Products(CategoryID)` to improve filtering and joining products by category.

10.4 Orders(OrderDate, Status)

A composite non-clustered index was created on `Orders(OrderDate, Status)` to improve queries filtering orders by date and status.

10.5 Products(StockQuantity)

A filtered non-clustered index was created on `Products(StockQuantity)` for products where `StockQuantity < 10`, supporting efficient low-stock queries.

Execution Plan Analysis

Execution plans were reviewed to evaluate query performance before and after indexing. Screenshots were captured as evidence of the execution-plan analysis.

The screenshots [41](#), and [42](#) are included as supporting evidence for the indexing and execution-plan section.

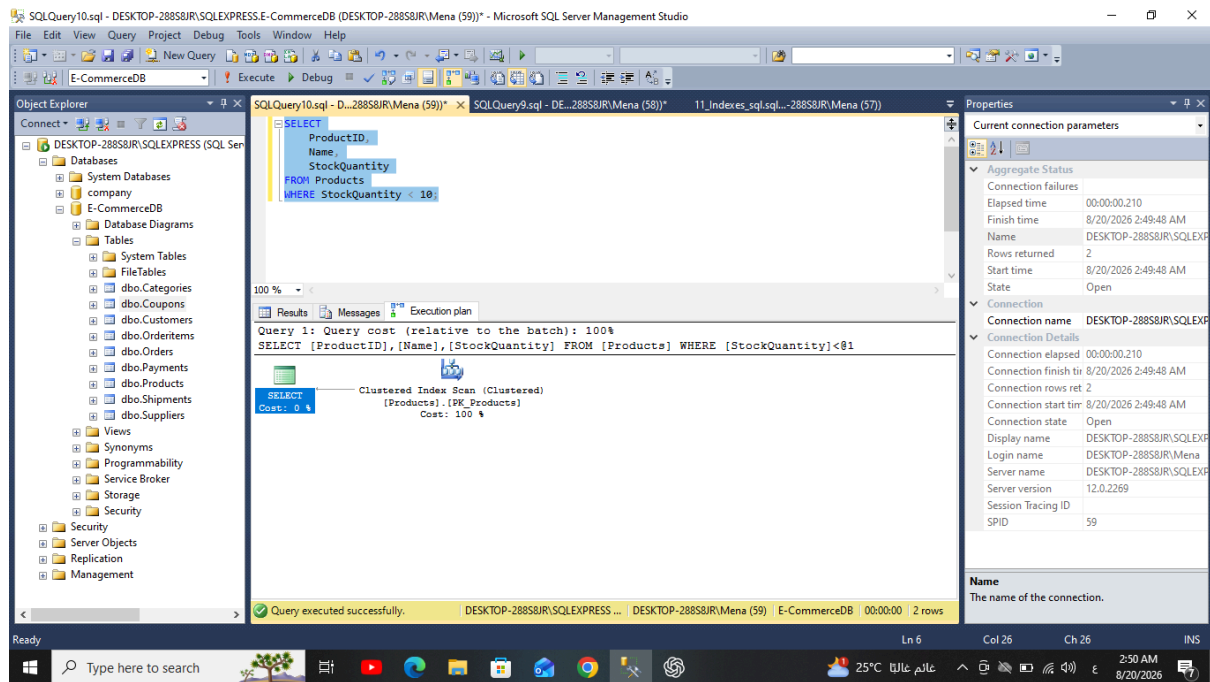
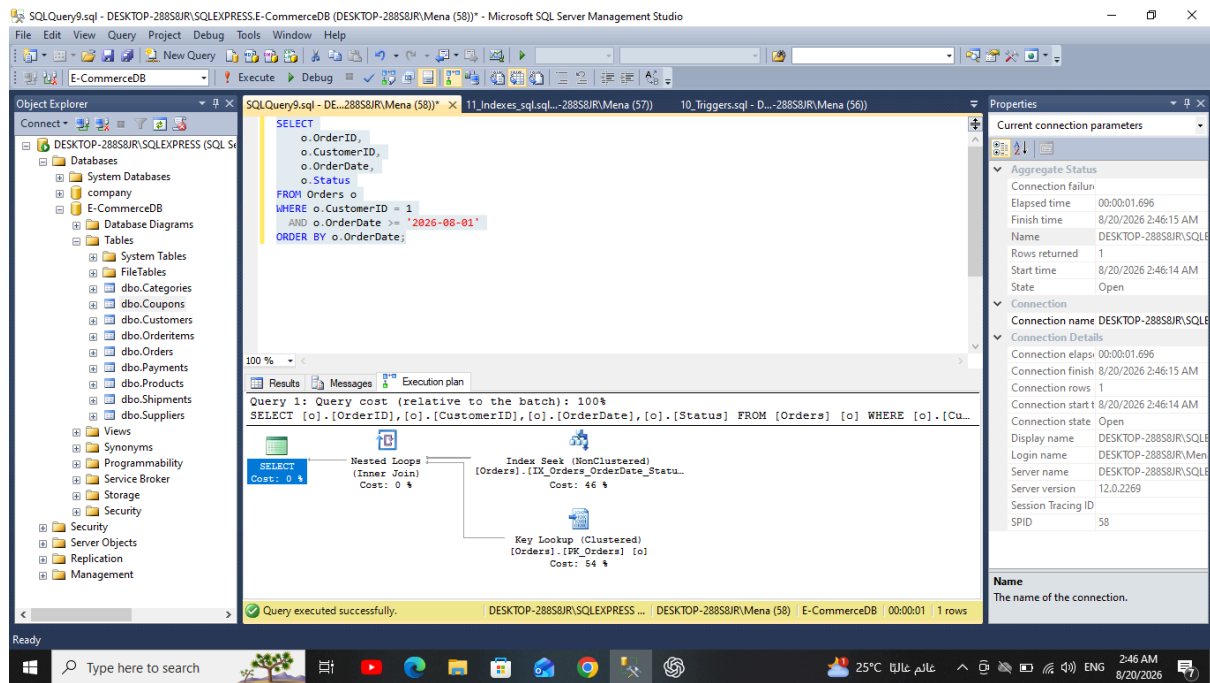


Figure 3. Execution plan analysis before and after indexing.

11.Power BI Dashboard

After completing the database implementation, functions, stored procedures, triggers, and indexes, a Power BI dashboard was created to provide a clear visual overview of the e-commerce system.

The dashboard presents key information about orders, payments, customers, and inventory, along with product categories, order statuses, top-selling products, low-stock products, and the overall stock level.

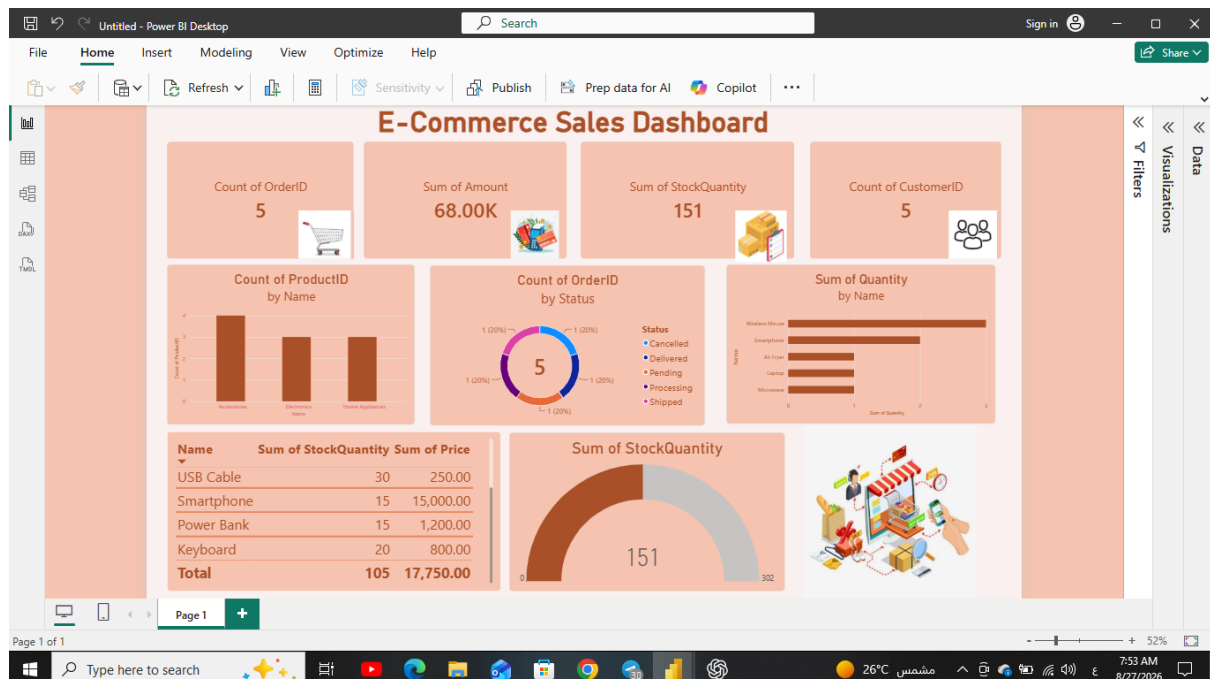


Figure 4: E-Commerce Sales Dashboard

This dashboard provides an interactive visual summary of the e-commerce database and helps users quickly understand the current sales, customer, order, and inventory information.

12. Conclusion

The E-Commerce Database project successfully implements a complete database system for managing customers, products, orders, payments, shipments, suppliers, and categories.

The project includes database tables with appropriate primary keys, foreign keys, constraints, and default values. It also includes functions, stored procedures, triggers, and indexes to improve data management, automation, integrity, and performance.

SQL queries and execution plans were used to verify and optimize the database performance. Finally, a Power BI dashboard was created to provide a clear visual overview of the main e-commerce data and support better analysis and decision-making.

Overall, the project demonstrates the practical application of SQL Server database design, programming, optimization, and data visualization in an e-commerce environment.

